



LightUP
Your Potential

CHƯƠNG TRÌNH ĐÀO TẠO DOMAIN BACKEND

Lesson 2

Indexing Database



Biên soạn bởi Hội đồng chuyên môn - Domain Backend



GIẢNG VIÊN



Phan Hồng Linh

Project Manager

Data & Solution Architect Department

MỤC LỤC



01. Giới thiệu về Index

02. Cấu trúc Index

03. Bình pháp đánh Index

1

GIỚI THIỆU VỀ INDEX



Khái niệm index



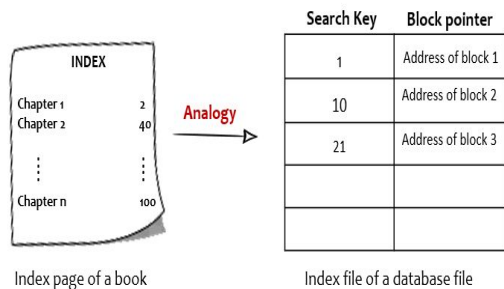
Lợi ích & Hạn chế



Các loại index cơ bản



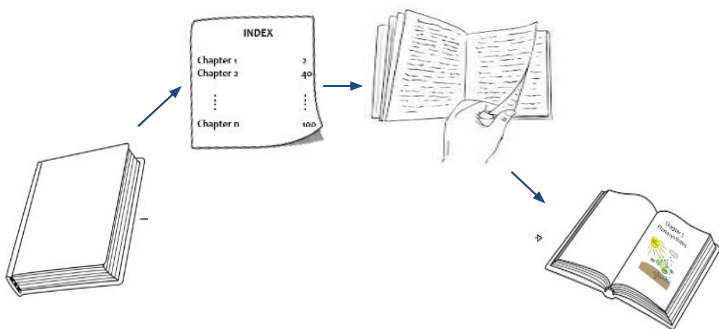
Khái niệm index



Index là một cấu trúc dữ liệu của database dùng để tăng hiệu suất truy vấn.

Index tương tự phần mục lục của cuốn sách.

Để tìm kiếm một nội dung trong cuốn sách nhanh hơn, ta tìm kiếm ở phần mục lục trước





Định nghĩa

* * * * *
 # | | | | | | |
 # | | | | | | ----- years (empty or 1970–2099)
 # | | | | | ----- day of week (1–7 or SUN–SAT)
 # | | | | ----- month (0–11 or JAN–DEC)
 # | | | ----- day of month (1–31)
 # | | ----- hours (0–23)
 # | ----- minutes (0–59)
 # ----- seconds (0–59)



Một chuỗi gồm sáu hoặc 7 thành phần con được sử dụng mô tả chi tiết về tác vụ lập lịch

0 15 10 ? * 6L 2022–2025
 Tác vụ sẽ chạy vào 10:15:00 AM thứ 6 cuối cùng mỗi tháng vào các năm 2022, 2023, 2024 and 2025.



Giới thiệu về Index

INDEX	TABLE
E00127	Tyler Bennett E10297
E01234	John Rappl E21437
E03033	George Woltman E00127
E04242	Adam Smith E63535
E10001	David McClellan E04242
E10297	Rich Holcomb E01234
E16398	Nathan Adams E41298
E21437	Richard Potter E43128
E27002	David Motsinger E27002
E41298	Tim Sampair E03033
E43128	Kim Arlich E10001
E63535	Timothy Grove E16398



Index cần vùng nhớ riêng và lưu trữ tách biệt với dữ liệu gốc.



Tạo index không làm thay đổi dữ liệu gốc:

- Tạo ra 1 bản copy của dữ liệu được đánh index và link đến dữ liệu gốc.
- Là một dạng thiết kế dư thừa dữ liệu trong database. Đánh đổi giữa hiệu năng và không gian lưu trữ.



Index được implement ở tầng storage engine

- Tùy thuộc vào database, storage engine hỗ trợ các loại index & cách implement khác nhau



Lợi ích của Index



- Tăng tốc độ các câu truy vấn (SELECT) dữ liệu (1)

- Giảm tải server và thao tác I/O xuống disk cho truy vấn dữ liệu lớn.

(1): Lưu ý: index giúp các truy vấn dạng tìm kiếm dữ liệu nhanh hơn (select nói chung). Điều này còn đúng cho cả các trường hợp UPDATE dữ liệu với điều kiện (WHERE) sử dụng index để tìm kiếm bản ghi cần cập nhật. (VD: UPDATE ... WHERE $x = 1$ với x thì câu lệnh UPDATE này sẽ nhanh hơn nếu x có đánh index)



Hạn chế của Index



- Thêm không gian lưu trữ cho index (disk, memory)

- Các lệnh cập nhật dữ liệu (insert, update, delete) chậm hơn do bảng phải maintain thêm index.

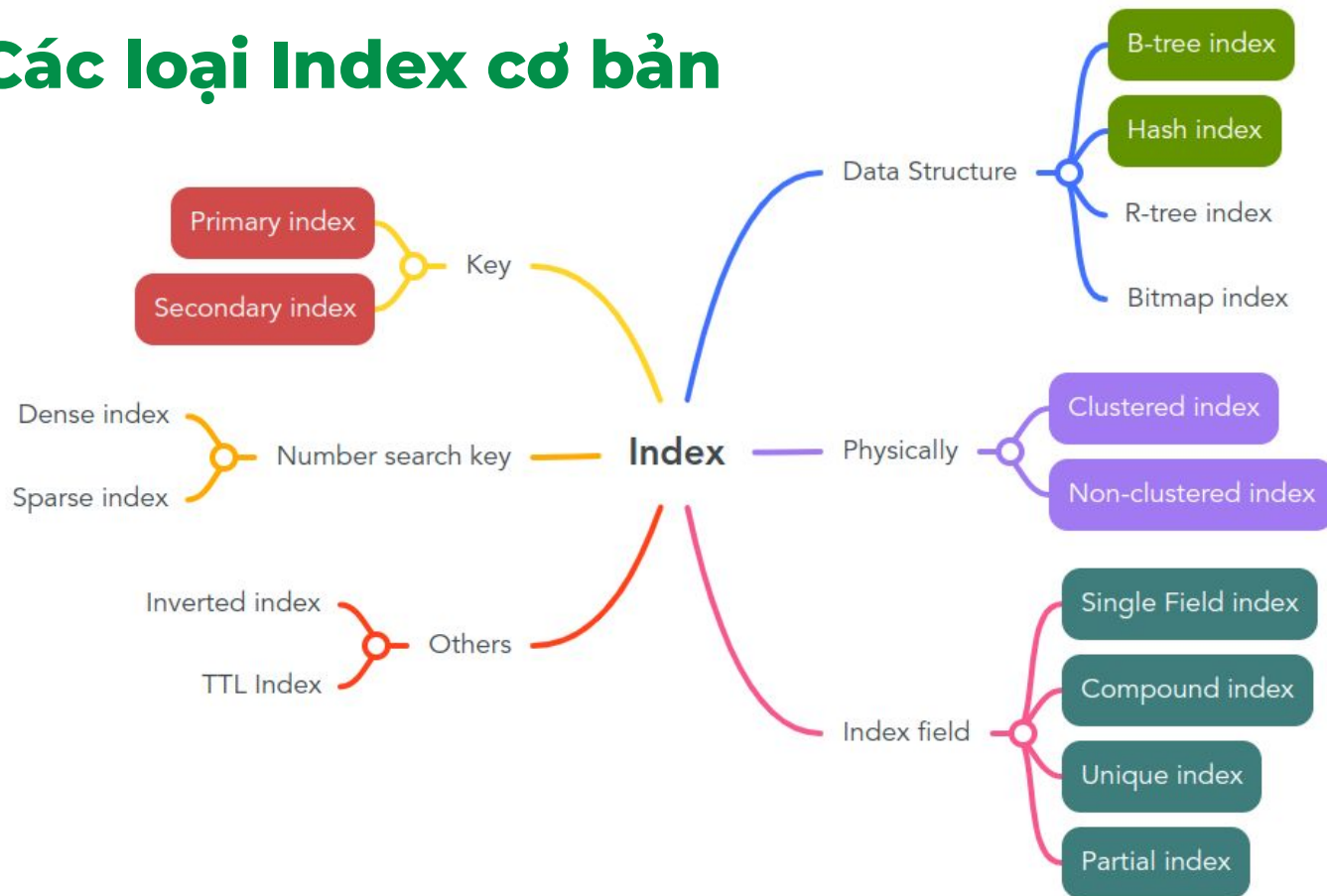
- Số lượng index lớn có thể làm chậm hệ thống.

- Có thể gây nhiễu kế hoạch thực thi câu truy vấn và nhầm lẫn khi lựa chọn index

Question: Tôi sử dụng ORM thì có cần quan tâm tới index?



Các loại Index cơ bản



2

CẤU TRÚC DỮ LIỆU INDEX



B-tree Index



Hash Index



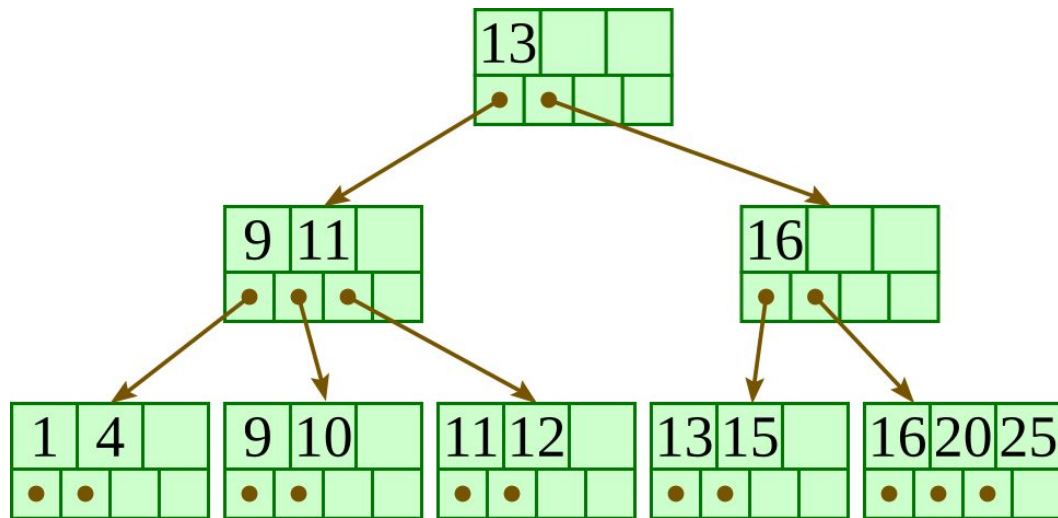
**Các tình huống sử dụng
index**



B-tree index

Là một cấu trúc dữ liệu dạng cây cân bằng. (Balanced-tree not Binary-tree)

- Mỗi node chứa n giá trị và chứa con trỏ tới các node dưới.
- Dữ liệu ở các node lá được sắp xếp từ bé đến lớn.
- Mỗi node đều chứa con trỏ tới dữ liệu gốc.



Visualization B-tree: <https://www.cs.usfca.edu/~galles/visualization/BTree.html>



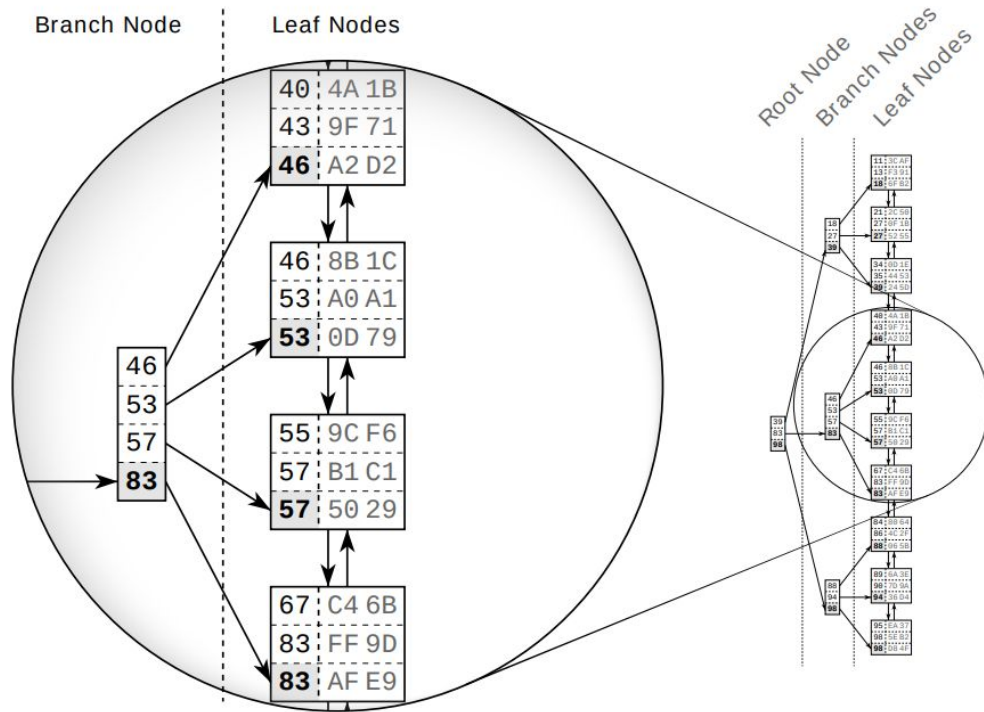
B+-tree index

Giữa các node lá có các liên kết dạng double linked list với nhau.

Mỗi node lá được lưu trữ trong 1 block (page): đơn vị lưu trữ nhỏ nhất của CSDL.

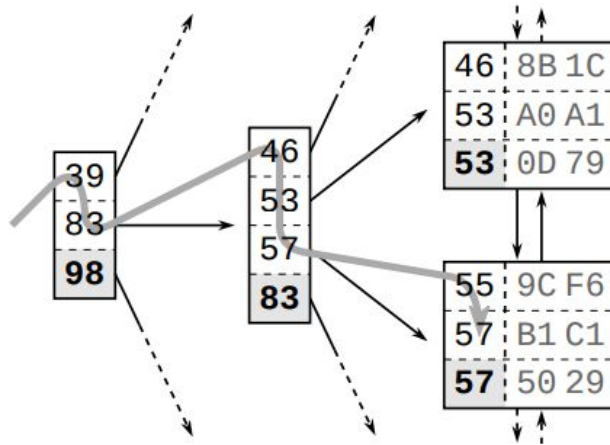
Block size phụ thuộc vào storage engine.

Trong MySQL (InnoDB), block size của index mặc định là 16kB





Tìm kiếm trên B+-tree



Index Leaf Nodes
(sorted)

column 2
ROWID

11	3C AF
13	F3 91
18	6F B2

21	2C 50
27	0F 1B
27	52 55

34	0D 1E
35	44 53
39	24 5D

Table
(not sorted)

column 1
column 2
column 3
column 4

A	34	1	2
A	27	5	9

A	39	2	5
X	21	7	2

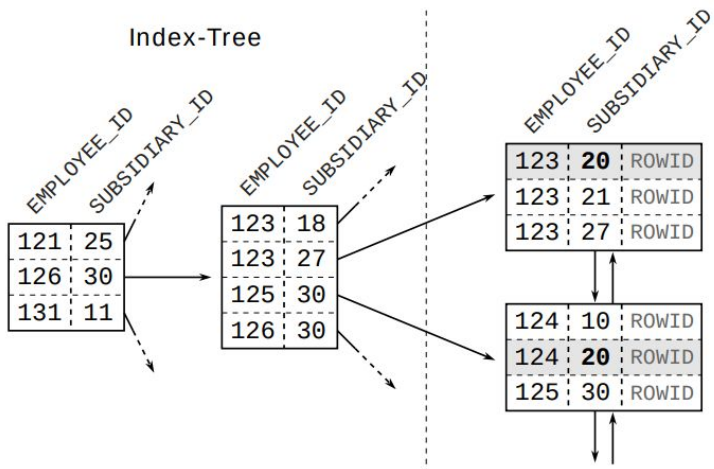
A	11	1	6
---	----	---	---

A	35	8	3
X	27	3	2

A	18	3	6
A	13	7	4



Các loại query có thể sử dụng B-tree index



Match full value

- ✓ WHERE Employee_id = 123
- ✓ WHERE Subsidiary_id = 20 AND Employee_id = 123

Match leftmost prefix

- ✓ WHERE Employee_id = 123
- ✗ WHERE Subsidiary_id = 20

Match range condition

- ✓ WHERE Employee_id >= 123 AND Employee_id < 131
- ✗ WHERE Subsidiary_id > 20

Match exactly in left and range in right columns

- ✓ WHERE Employee_id = 123 AND Subsidiary_id < 20
- ✗ WHERE Employee_id > 123 AND Subsidiary_id = 20



Các loại query có thể sử dụng B-tree index

LIKE 'WI%ND'

WIAW
WIBLQQNPUA
WIBYHSNZ
WIFMDWUQMB
WIGLZX
WIH
WIHTFVZNLC
WIJYAXPP
WINAND
WINBKYDSKW
WIPOJ
WISRGPK
WITJIVQJ
WIW
WIWGPJMQGG
WIWKHLBJ
WIYETHN
WIYJ

LIKE 'WIN%D'

WIAW
WIBLQQNPUA
WIBYHSNZ
WIFMDWUQMB
WIGLZX
WIH
WIHTFVZNLC
WIJYAXPP
WINAND
WINBKYDSKW
WIPOJ
WISRGPK
WITJIVQJ
WIW
WIWGPJMQGG
WIWKHLBJ
WIYETHN
WIYJ

LIKE 'WINA%'

WIAW
WIBLQQNPUA
WIBYHSNZ
WIFMDWUQMB
WIGLZX
WIH
WIHTFVZNLC
WIJYAXPP
WINAND
WINBKYDSKW
WIPOJ
WISRGPK
WITJIVQJ
WIW
WIWGPJMQGG
WIWKHLBJ
WIYETHN
WIYJ



LIKE Operator (Left mode prefix)

✓ WHERE Name like 'WI%ND'

✗ WHERE Name like '%WIND%'



Các loại query có thể sử dụng B-tree index

```
SELECT id, pkg_order
FROM `new_package_logs`
WHERE `pkg_order` IN
(2008401890)
AND `action` IN
('PrepaymentBill');
```

=>> Output:
1 records

```
SELECT id, pkg_order, old_value
FROM `new_package_logs`
WHERE `pkg_order` IN
(2008401890)
AND `action` IN
('PrepaymentBill');
```

=>> Output:
2 records

Index-only queries (Covering Index)

✓ Select Subsidiary_id FROM Employee
WHERE Employee_id > 123
Explain format=json: *using_index: true*

Index merge

✓ Select id, pkg_order, desc from
new_package_logs WHERE id = 100 OR
pkg_order = 2008401890

Sorting/Group by

✓ Select id, pkg_order, created FROM
packages ORDER BY id DESC Limit 10



Các loại query có thể sử dụng B-tree index

```
{
  "access_type": "index_merge",
  "possible_keys": [
    "IDX_PACKAGE",
    "IDX_ACTION"
  ],
  "key": "intersect(IDX_PACKAGE,IDX_ACTION)",
  "key_length": "9,152",
  "rows_examined_per_scan": 1,
  "rows_produced_per_join": 1,
  "filtered": "100.00",
  "using_index": true,
  "cost_info": {
    "read_cost": "4.81",
    "eval_cost": "0.20",
    "prefix_cost": "5.01",
    "data_read_per_join": "1K"
  },
  "used_columns": [
    "id",
    "pkg_order",
    "action"
  ],
}
```

Index-only queries (Covering Index)

✓ Select Subsidiary_id FROM Employee
WHERE Employee_id > 123
Explain format=json: *using_index: true*

Index merge

✓ Select id, pkg_order, desc from
new_package_logs WHERE id = 100 OR
pkg_order = 2008401890

Sorting/Group by

✓ Select id, pkg_order, created FROM
packages ORDER BY id DESC Limit 10



Các loại query có thể sử dụng B-tree index

```
CREATE TABLE `users` (
  `id` INT(11) NOT NULL AUTO INCREMENT,
  `username` VARCHAR(255) NOT NULL,
  `fullname` VARCHAR(255),
  `group_id` INT(11) DEFAULT NULL,
  `birthday` DATE,
  `created` DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `username` (`username`),
  KEY `IDX_GROUP_ID` (`group_id`),
) ENGINE=INNODB CHARACTER SET utf8 COLLATE utf8_unicode_ci;
```

```
EXPLAIN SELECT id, fullname FROM users where group_id = 15;
```

Filter Rows:

Export: Wrap Cell Content:

select_type	table	partition	type	possible keys	key	key_len	ref	rows	filtered	Extra
SIMPLE	users		NULL	ref	IDX_GROUP_ID	5	const	357	100.00	PULL

key_len = 4 bytes (int) + 1 byte (NULL)

Function Index

- ✗ SELECT id, fullname FROM users WHERE YEAR(birthday) + 2 > YEAR('1999-12-01')
- ✓ SELECT id, fullname FROM users WHERE birthday > YEAR('1999-12-01') - 2
- ✗ SELECT id, fullname FROM users WHERE UPPER(fullname) = UPPER('Phan Hong Linh');
- ✓ CREATE INDEX idx_1 ON users (UPPER(fullname)); (Oracle, MySQL 8.0,...)

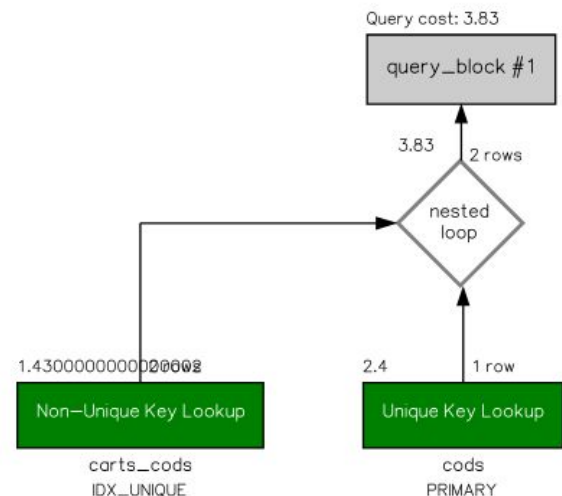
Indexing with NULL value

MySQL (InnoDB) cần thêm ít nhất 1 byte để đánh index cho các cột NULLABLE (Check *key_length* ở Explain)

- ✓ Để NOT NULL nếu có thể với các cột được đánh index



Các loại query có thể sử dụng B-tree index



JOIN

EXPLAIN

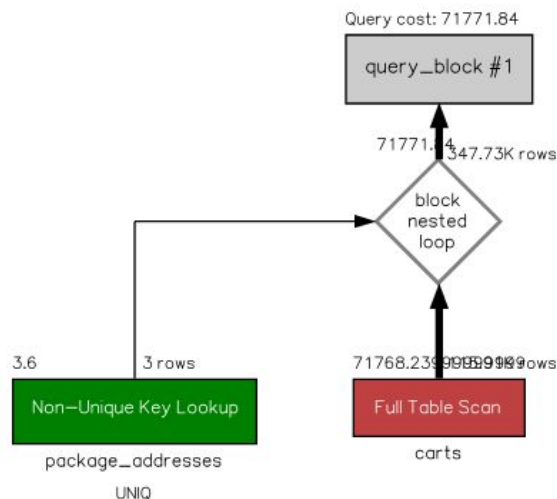
```

SELECT carts_cods.cart_id, carts_cods.cod_id, cods.`fullname`
FROM carts_cods
INNER JOIN cods ON cods.id = carts_cods.cod_id
WHERE carts_cods.cart_id = '5edf24b0-b978-4eeb-94d8-46190a0a020b' ;
  
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	carts_cods	(NULL)	OK	ref	IDX_UNIQUE,IDX_COD_ID,IDX_CART_ID	IDX_UNIQUE	109	const	2	100.00 Using where; Using index
1	SIMPLE	cods	(NULL)	OK	eq_ref	PRIMARY	PRIMARY	108	ghtk.carts_cods.cod_id	1	100.00 (NULL)



Các loại query có thể sử dụng B-tree index



Tại sao không sử dụng được index ở trong tình huống join này? (DB ghtk)

```

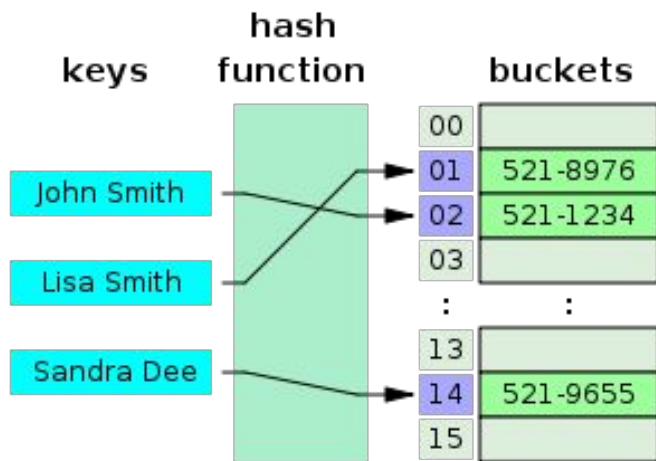
SELECT pa.`package_order`, c.`alias`, c.`first_cod_id`
FROM `package_addresses` pa
INNER JOIN carts c ON c.id = pa.cart_id
WHERE pa.package_order = 1510775540 ;
  
```

<https://saiyan.ghk.vn/excution-plan>

table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
pa	(NULL)	OK	ref	UNIQ	UNIQ	8	const	3	100.00 (NULL)
c	(NULL)	OK	ALL	(NULL)	(NULL)	(NULL)	(NULL)	115909	100.00 Using where; Using join buffer (Block Nested Loop)



Hash Index



CTDL Hash Index dựa trên kỹ thuật hashing, xây dựng một Hash Table để lưu trữ và tìm kiếm dữ liệu

- Hash Table sử dụng hash function để tìm kiếm một index vào một mảng các buckets.
- Hệ quản trị CSDL support Hash Index: MySQL (Memory), Oracle, PostgreSQL



Tính chất của Hash Index



Tốc độ truy vấn nhanh với so sánh bằng “=”



Không thể sử dụng trong các trường hợp so sánh khoảng, sorting, group by



Tốn nhiều không gian hơn với B-tree.
(Cần lưu hash value,...)



Vấn đề đụng độ, không nên sử dụng cho các cột có nhiều giá trị lặp

3

***BINH PHÁP
ĐÁNH INDEX***

Các chiến lược đánh index



Vận dụng index vào thực tế

01

Lựa chọn cột đánh index (Index Selectivity)

```
CREATE TABLE `users` (
  `id` INT(11) NOT NULL AUTO INCREMENT,
  `username` VARCHAR(255) NOT NULL,
  `fullname` VARCHAR(255) DEFAULT NULL,
  `group_id` INT(11) DEFAULT NULL,
  `status_id` INT(11) NOT NULL DEFAULT '1',
  `created` DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `username` (`username`),
  KEY `IDX_GROUP_ID` (`group_id`),
  KEY `IDX_STATUS` (`status_id`),
) ENGINE=INNODB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci;
```

SHOW INDEX FROM users;

Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality
0	PRIMARY	1	id	A	32042
0	username	1	username	A	32042
1	IDX_GROUP_ID	1	group_id	A	19
1	IDX_STATUS	1	status_id	A	2



Phụ thuộc vào các câu truy vấn sẽ sử dụng
 Hãy liệt kê hết các câu truy vấn sẽ dùng trước
 khi quyết định đánh index gì cho bảng đó.



**Không nên đánh index các cột có ít
 distinct value (cardinality thấp)**

VD: status, action, is_active,...

Case đặc biệt: status có phân phối dữ liệu
 lệch hẳn về 1 giá trị. -> Index vẫn có giá trị.

02

Đánh prefix Index

```
CREATE TABLE `users` (  
  `id` INT(11) NOT NULL AUTO_INCREMENT,  
  `username` VARCHAR(255),  
  `fullname` VARCHAR(255),  
  `group_id` INT(11) DEFAULT NULL,  
  `status_id` INT(11) NOT NULL DEFAULT '1',  
  `created` DATETIME DEFAULT CURRENT_TIMESTAMP,  
  `iam_id` VARCHAR(255) COMMENT 'id của hệ thống GHTK ID',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `username` (`username`),  
  KEY `iam_id` (`iam_id`(10)),  
  KEY `IDX_GROUP_ID` (`group_id`),  
  KEY `IDX_STATUS` (`status_id`)  
) ENGINE=INNODB CHARACTER SET utf8 COLLATE utf8_unicode_ci;
```



Lựa chọn prefix index:

- Đủ dài để sử dụng index hiệu quả.
- Đủ ngắn để tiết kiệm không gian lưu trữ.

Cột iam_id varchar(255):

- ❖ Đánh Index full length: Key length = $255 \times 3 + 1$ (varchar) + 2 (Index varchar with NULL) = 768 bytes
- ❖ Đánh prefix index: Key length = $10 \times 3 + 1$ (varchar) + 2 (Index varchar with NULL) = 33 bytes

03 Đánh index trên nhiều cột (Multi-column Index)

```
CREATE TABLE `users` (  
  `id` INT(11) NOT NULL AUTO_INCREMENT,  
  `username` VARCHAR(255) NOT NULL,  
  `fullname` VARCHAR(255) DEFAULT NULL,  
  `group_id` INT(11) DEFAULT NULL,  
  `status_id` INT(11) NOT NULL DEFAULT '1',  
  `station_id` INT(11) NOT NULL,  
  `created` DATETIME DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `username` (`username`),  
  KEY `IDX_GROUP_ID` (`group_id`),  
  KEY `IDX_STATION_GROUP` (`station_id`, `group_id`),  
) ENGINE=INNODB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci;
```

```
SELECT * FROM users WHERE station_id = 4558 AND group_id = 15;
```

```
SELECT * FROM users WHERE station_id = 4558;
```



Đánh index kết hợp nhiều cột thay vì đánh trên từng cột riêng lẻ.

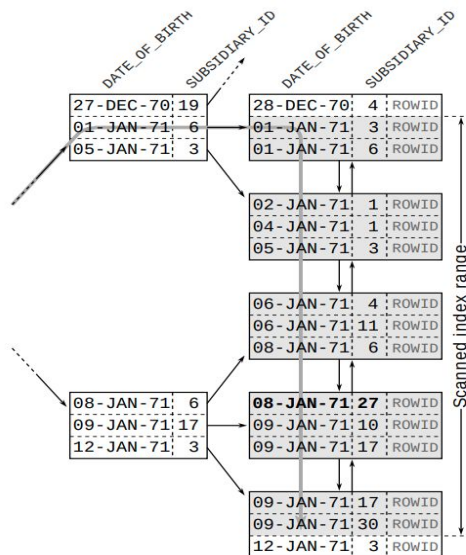
- Với query kết hợp điều kiện trên nhiều cột, nếu đánh index trên 1 cột không hiệu quả, cần cân nhắc đánh kết hợp trên nhiều cột.



Với tình huống đánh index trên từng cột lẻ MySQL vẫn có thể sử dụng cả 2 index bằng Index Merge nhưng chi phí cho câu query sẽ lớn hơn.

04

Lựa chọn thứ tự cột khi đánh multi-column index



```
SELECT * FROM users WHERE subsidiary_id = 4
AND date_of_birth between '06-Jan-71' AND '09-Jan-71'
```

✓ CREATE INDEX idx1 ON users(subsidiary_id, date_of_birth);
 ✗ CREATE INDEX idx2 ON users(date_of_birth, subsidiary_id);



Lựa chọn thứ tự cột hợp lý để có thể tận dụng left mode prefix.

- Khi đánh index thứ tự (A,B) thì chúng ta không cần đánh index cho riêng cột A theo quy tắc left mode prefix.



Lựa chọn cột có tính selective cao hơn đứng bên trái để sử dụng index filter hiệu quả. (Đa số trường hợp)



Để cột query dạng range sang bên phải.

- Do tính chất của B-tree, nếu cột có query dạng khoảng đứng trước thì cột còn lại sẽ không được sử dụng trong index.

05

Clustered index

```
CREATE TABLE `packages` (
  `id` CHAR (36) COLLATE utf8_unicode_ci NOT NULL,
  `order` BIGINT (20) NOT NULL,
  `shop_id` CHAR (36) COLLATE utf8_unicode_ci NOT NULL,
  `pick_address_id` INT (11) DEFAULT NULL,
  `deliver_address_id` INT (11) DEFAULT NULL,
  `ship_money` INT (10) UNSIGNED DEFAULT 0,
  PRIMARY KEY (`order`),
  UNIQUE KEY `IDX_PACKAGE_ID` (`id`),
  KEY `IDX_CUR_BAG_ORDER` (`cur_bag_order`),
) ENGINE = INNODB DEFAULT CHARSET = utf8 COLLATE = utf8_unicode_ci
```

Name	Engine	Index_length	Data_free	Rows	Avg_row_length	Data_length
packages	InnoDB	12,401,754,112	3,489,660,928	488,909	5,555	30,495,752,192
deliver_packages	InnoDB	1,064,878,080	603,979,776	637,375	249	408,961,024
package_logs	InnoDB	32,768	314,572,800	310	52	16,384
tmp_station_of_pkgs	InnoDB	132,153,344	281,018,368	266,006	394	104,923,136
key_verify_logs	InnoDB	32,768	170,917,888	77	212	16,384
picking_packages	InnoDB	37,879,808	132,120,576	247,991	169	42,008,576
auto_weighing_logs	InnoDB	6,886,883,328	94,371,840	645,687	62	4,075,470,848



Clustered index - Lưu trữ dữ liệu ở mức vật lý

- Trong InnoDB, clustered index thường là primary key. Dữ liệu của bảng được lưu ở các node lá với block size mặc định là 16kB. Hầu hết các câu truy vấn đều có bước tìm kiếm dữ liệu qua primary key.



Lựa chọn Primary key quyết định sự sống còn của table

- Giảm thiểu disk I/O, truy cập nhanh, covering index
- Một số kiểu primary key: Auto Increment, UUID, Hash value, Snowflake ID,...



Thứ tự insert của primary key ảnh hưởng tới hiệu năng của bảng.

- Insert dữ liệu random gây phân mảnh bảng, dữ liệu trên pages bị sắp xếp lại liên tục làm tăng I/O, kém hiệu quả với select range,...

06

Covering Indexes

```
    "access_type": "index_merge",
    "possible_keys": [
      "IDX_PACKAGE",
      "IDX_ACTION"
    ],
    "key": "intersect(IDX_PACKAGE,IDX_ACTION)",
    "key_length": "9,152",
    "rows_examined_per_scan": 1,
    "rows_produced_per_join": 1,
    "filtered": "100.00",
    "using_index": true,
    "cost_info": {
      "read_cost": "4.81",
      "eval_cost": "0.20",
      "prefix_cost": "5.01",
      "data_read_per_join": "1K"
    },
    "used_columns": [
      "id",
      "pkg_order",
      "action"
    ],
  },
```

**Sử dụng index để “cover” toàn bộ câu truy vấn**

- Dữ liệu trong index không chỉ sử dụng ở điều kiện WHERE mà còn có thể dùng để trả kết quả cho câu truy vấn, giảm I/O

```
SELECT id, pkg_order
FROM `new_package_logs`
WHERE `pkg_order` IN (2008401890)
AND `action` IN
('PrepaymentBill');
```

=>> Output:
1 records

```
SELECT id, pkg_order, old_value
FROM `new_package_logs`
WHERE `pkg_order` IN (2008401890)
AND `action` IN
('PrepaymentBill');
```

=>> Output:
2 records

07

Loại bỏ index dư thừa

```
select * from sys.`schema_unused_indexes`;
```

object_schema	object_name	index_name
labeling	emp_positions	IDX_DEPARTMENT
labeling	emp_profiles	IDX_EMP_POSITION
labeling	emp_profiles	IDX_UID
labeling	emp_profiles	IDX_DUTY
labeling	ghtk_users	IDX_STATION_ID
labeling	ghtk_users	IDX_GROUP_ID
labeling	ghtk_users	IDX_STATION_STT
labeling	stations	IDX_PRID
labeling	stations	IDX_MODIFIED
labeling	stations	IDX_DISTRICT
labeling	stations	IDX_TC_STATION
labeling	users_work_types	IDX_USER_ID
labeling	users_work_types	IDX_WORK_TYPE_ID
labeling	users_work_types	IDX_STATUS
labeling	users_work_types	IDX_USER_ID_STATUS



Lựa chọn thứ tự cột phù hợp trong multi-column index để loại bỏ những index dư thừa.



Loại bỏ những index không bao giờ được sử dụng, index lặp.

08 Mượn giá trị ánh xạ để dùng index

```
CREATE TABLE `query_pattern` (  
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `query` TEXT NOT NULL COMMENT 'query pattern',  
  `hash` CHAR(32) CHARACTER SET ASCII NOT NULL  
    COMMENT 'md5 hash of query pattern string',  
  `db_user_id` INT NOT NULL COMMENT 'user id of database',  
  `group_id` INT DEFAULT NULL,  
  `schema_id` INT DEFAULT NULL,  
  `issue_id` INT DEFAULT NULL,  
  `first_seen` DATETIME DEFAULT CURRENT_TIMESTAMP,  
  `last_seen` DATETIME DEFAULT CURRENT_TIMESTAMP,  
  `created` DATETIME DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `IDX_HASH` (`hash`(10)),  
  KEY `IDX_GROUP_CREATED` (`group_id`, `created`),  
  KEY `IDX_ISSUE` (`issue_id`)  
) ENGINE=INNODB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```



Mượn giá trị hash để so sánh và đánh index cho dữ liệu dạng text, varchar lớn

09 Đánh index một phần (Partial Index)

```
CREATE TABLE `task_assign` (  
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `task_id` BIGINT NOT NULL,  
  `status` TINYINT DEFAULT 0 COMMENT '0: new, 1: done',  
  `created` DATETIME DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `IDX_STATUS`,  
) ENGINE=INNODB ;
```

```
CREATE INDEX IDX_STATUS ON task_assign(`status`)  
WHERE `status` = 0;
```



Đánh index trên một phần dữ liệu giúp giảm không gian lưu trữ.

Lưu ý: MySQL 5.7 trở xuống không hỗ trợ partial index

10 Cập nhật index statistic



Dữ liệu Index và bảng có thể bị hiện tượng phân mảnh (fragmentation)

SHOW TABLE STATUS, SHOW INDEX để kiểm tra vấn đề phân mảnh.



Chạy **ANALYZE TABLE, OPTIMIZE TABLE** định kỳ để cập nhật Index statistics và hạn chế phân mảnh bảng.

Hành động này được thực hiện bởi chuyên gia và cần review kỹ!

11 Tuyệt chiêu cuối: NO INDEX!



Tam Thập Lục Kế

Tẩu Vi Thượng



Index không phải là chìa khóa vạn năng để giải quyết mọi vấn đề!



Bảng có kích thước nhỏ không nhất thiết phải đánh index.



Thiết kế schema phù hợp, tối ưu query, logic để câu truy vấn đơn giản, bảng không phải maintain quá nhiều index.



Chuyển qua sử dụng các kỹ thuật khác:

- Học cách buông bỏ cũng là một sự lựa chọn.
- Sử dụng database phù hợp cho truy vấn dạng OLAP
- Các kỹ thuật CDC, build tmp table, partition,...

TỔNG KẾT



Thuật ngữ index rất rộng và phụ thuộc vào CSDL bạn đang sử dụng!



Cần đánh index đúng cách để tối ưu hiệu năng hệ thống!



Tùy vào bài toán và ngữ cảnh cụ thể để lựa chọn chiến lược đánh index khác nhau!



LightUP
Your Potential

CHƯƠNG TRÌNH ĐÀO TẠO DOMAIN BACKEND

TRÂN TRỌNG CẢM ƠN

Chúc bạn có trải nghiệm học tập và làm việc tốt tại GHTK

